



Lab Assignment 01

Aim:

Implement multithreaded client/server process communication using RMI.

Objective:

- ↳ Multi threading
- ↳ How Remote Method Invocation works
- ↳ How RMI allows objects to invoke methods on remote object
- ↳ How to write Distributed Object Application

Theory:

Tools used -

1. Windows
2. Linux
3. Java language.

RMI (Remote Method Invocation) is used for distributed object references system. A distributed object is an object which publishes its interfaces on other ~~surfaces~~ machines. A remote object is a distributed object whose state is encapsulated. Stub and Skeleton are two objects used to communicate with remote object.

Stub: Stub is a gateway for client program which is used to communicate with skeleton object by establishing a connection between them.



Skeleton: provides on server program which is used for passing the request from stub to the remote interface

Steps to run Java RMI applications

Creation of classes and interfaces for the problem statement

The steps involved are -

1. Create the remote interface
2. Provide the implementation
3. Create and run server applications
4. Create and run client applications

Result -

Successfully the Client server communication is done.



Lab Assignment - 02

Aim:

Develop any distributed application using CORBA to demonstrate object brokering

Objective:

Basic implementation of a Java/CORBA application using static invocations

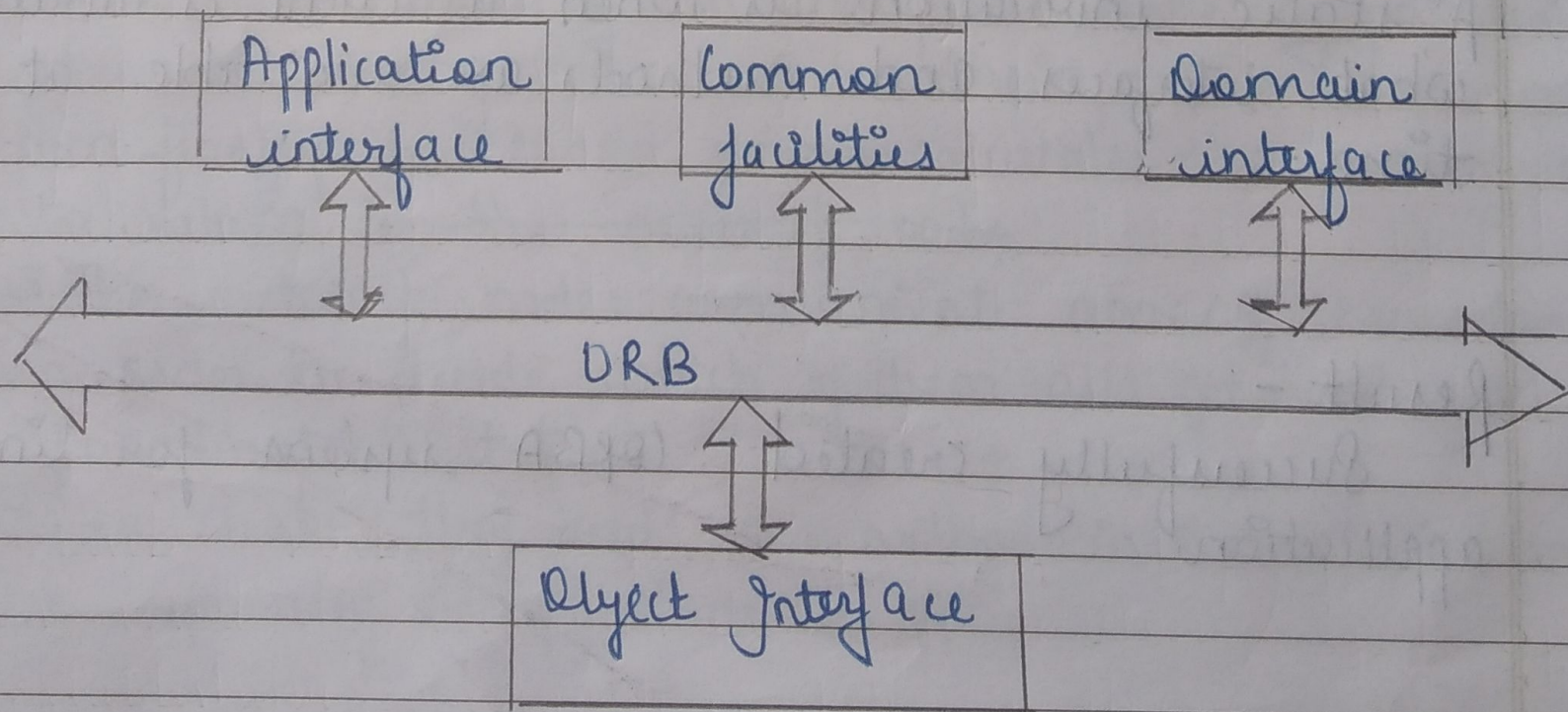
Theory:

Common Object Request Broker Architecture (CORBA) could be a specification of a regular design for middleware.

It is a client server software development model.

CORBA Reference Model -

The CORBA reference model known as Object Management Design (OMA) is shown below -





The DMA is itself a specification that defines a broad variety of services for building distributed client server applications.

Divide & conquer - The remote objects can be independently designed.

Increase reusability - It is often possible to design remote objects so that other systems can use them too.

Increase reuse - You may be able to reuse remote objects that others have created.

Design for flexibility - The broker objects can be updated as required or you can redirect the proxy to communicate with different objects.

The calculator application is a basic implementation of a Java / CORBA application using static invocations. A static invocation is when a client is aware of what interfaces and methods are available at compile time.

Result -

Successfully created CORBA system for timestamp application



Lab Assignment - 03.

Aim:

Develop a distributed system to find sum of N elements in an array by distributing N/n elements into n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate sums calculated at different processors.

Objective:

To learn the demonstration by displaying the intermediate sums calculated at different processors.

Theory:

- ↳ Library of routines that can be used to create parallel programs in C or Fortran 77.
- ↳ It allows users to build parallel applications by creating parallel processes and to reduce the time complexity of the program.
- ↳ parallel execution of subarrays is done by parallel processes running to calculate their partial sums and then finally the master process calculates the sum of them to return to the network node.
- ↳ The network nodes communicate among themselves in order to decide which of them will get into the "coordinator" state.
- ↳ For that, they need some method in order to break the symmetry among them.



Installation of OPENMPI

1. Download `openmpi-4.1.4.tar.bz2`
2. Goto terminal
3. update using `sudo apt-get update`, `sudo install gcc`.
4. Goto the directory which contains the downloaded file
5. Extract the files
6. The directory `openmpi-4.1.4` is created
7. Configure, compile and install
8. Now `openmpi` folder is created in `opt` folder.
9. Update the path and environmental variables
10. Compile the program
11. Execute the program.

Result -

Successfully implemented average number sum calculation



Lab Assignment - 04

Aim:

Implement Berkeley algorithm for clock synchronization.

Objective:

Ignoring significant outliers in calculation of average time difference.

Theory:

Berkeley's Algorithm is a clock synchronization technique used in distributed systems.

The algorithm assumes that each machine node in the network either doesn't have an accurate time source or doesn't possess a UTC server.

Algorithm -

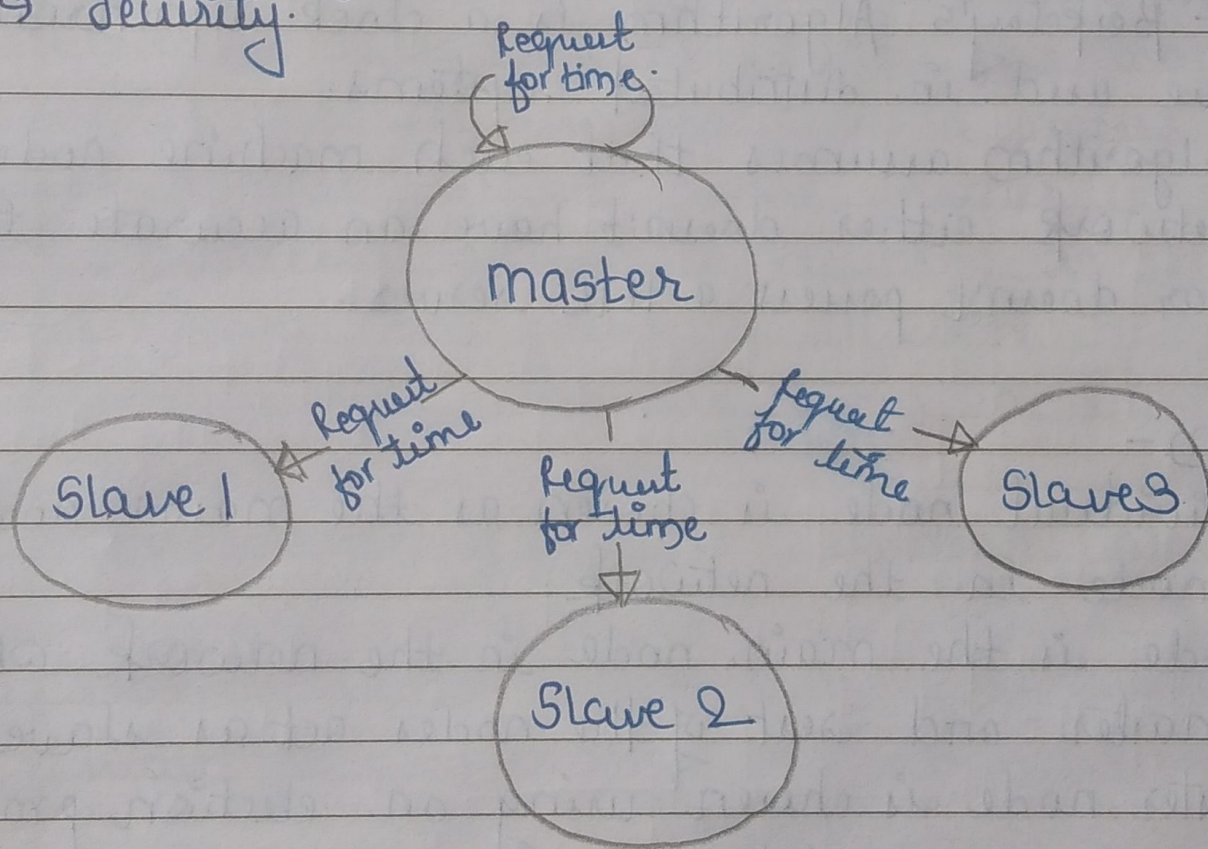
- ↳ An individual node is chosen as the master node from a pool node in the network.
- ↳ This node is the main node in the network which acts as a master and rest of the nodes act as slaves.
- ↳ The master node is chosen using an election process / leader election algorithm.
- ↳ Master node periodically ping slaves nodes and fetches clock time at them using Cristian's algorithm.
- ↳ Master node calculates the average time difference between all the clock times received and clock time given by master's system clock itself.
- ↳ This average time difference is added to the



current time at the master's system clock and broadcasted over the network

features of Berkeley's algorithm

- ↳ Centralized time coordinator
- ↳ clock adjustment
- ↳ Average calculation
- ↳ fault tolerance
- ↳ Accuracy
- ↳ Scalability
- ↳ Security



Result -

Maintaining the global time and distributing it to all the client machines



Lab Assignment - 05

Aim:

Implement token ring based mutual exclusion algorithm.

Objective:

To learn sequence number is used to distinguish old and current requests.

Theory:

Mutual exclusion is a concurrency control property which is introduced to prevent race conditions.

It is the requirement that a process can not enter its critical section while another concurrent process is currently present or executing in its critical section. i.e only one process is allowed to execute the critical section at any given instance of time.

Requirements of mutual exclusion algorithm

- ↳ No deadlock - Two or more sites should not endlessly wait for any message that will never arrive.
- ↳ No starvation - Every site who wants to execute critical section should get an opportunity to execute it in infinite time.
- ↳ Fairness - Each site should get a fair chance to execute critical section.



↳ fault tolerance - ~~each site should get a fair chance to execute critical section~~ In case of failure, it should be able to recognize it by itself in order to continue functioning without any any disruption

Solution to distributed mutual exclusion -

1. Token based algorithm

↳ A unique token is shared among all the sites
↳ If a site possesses the unique token, it is allowed to enter its critical section.

↳ This approach uses the sequence number to order requests for the critical section.

eg - Suzuki-Kasami's Broadcast algorithm.

2. Nontoken based approach

↳ A site communicates with other sites in order to determine which sites should execute critical section next.

↳ This approach uses timestamps instead of sequence number to order requests for critical section

eg - Lamport's Algorithm.

3. Quorum based approach

↳ Instead of requesting permission to execute critical section from all other sites, each site requests only a subset of sites which is called a quorum.

eg - Maekawa's algorithm.

Result -

Successfully communicate with multiple client machine from server.



Lab Assignment - DG.

Aim:

Implement Bully and Ring algorithm for leader election

Objective:

Coordinator that performs functions needed by other processes.

Theory:

↳ The bully algorithm is a type of election algorithm which is mainly used for choosing a coordinator. In a distributed system, we need some election algorithms such as bully and ring to get a coordinator that performs functions needed by other processes.

↳ Election algorithms select a single process from the processes that act as coordinator. A new process is selected when the selected coordinator process crashes due to some reasons.

↳ It assumes that each process has a unique priority number in the system, so the highest priority process will be taken first as a new coordinator. When the current coordinator process crashes, it elects a new process having the highest priority number.



The Bully election algorithm -

- ↳ Let's assume that P is a process that sends a message to the coordinator.
- ↳ It will assume that the coordinator process is failed when it doesn't receive any response from the coordinator within the time interval T .
- ↳ An election message will be sent to all the active processes by process P along with the highest priority number.
- ↳ If it will not receive any response within the time interval T , the current process P elects itself as a coordinator.
- ↳ After selecting itself as a coordinator, it again sends a message the process P is elected as their new coordinator to all the processes having lower priority.
- ↳ If process P will receive any response from another process Q within time T .
- ↳ It again waits for the time T to receive another response i.e. it has been elected as coordinator from process Q .
- ↳ If it doesn't receive any response within time T , it is assumed to have failed and the algorithm is restarted.

Result -

You have executed all the processes.



Lab Assignment - 07

Aim:

Create a simple web service and write any distributed application to consume the web service.

Objective:

Web service can be created as a collection of open protocols and standards for exchanging information among systems or applications.

Theory:

Service provider or publisher.

This is the provider of the web service. The service provider implements the service and makes it available on the internet or intranet.

Service requester or consumer.

This is any consumer of the web service. The requester utilizes an existing web service by opening a network connection and sending an XML request.

We will also write two web service requesters - one web-based consumer and another windows application-based consumer.

To test a web service, it must be published. A web service can be published either on an intranet or the internet. We will publish this web service on IIS running on a local machine.



- ↳ Open Start → Settings → Control Panel → Administrative tools
→ Internet services manager.
- ↳ Expand and right click on default web site; select New, Virtual Directory. The virtual Directory creation Wizard opens. Click next.
- ↳ The "Virtual Directory Alias" ~~screen~~ screen opens. Type the virtual directory name. Click Next.
- ↳ The "Web site content Directory" screen opens.
- ↳ Enter the directory path name for the virtual directory. Click next.
- ↳ The "access permission" screen opens. Change the settings as per your requirement.
- ↳ Click next button
- ↳ Click finish.

Testing the web service

Writing web service is easy in the .Net framework. Writing web service consumers is also easy in the .Net framework; however, it is a bit more involved.

As said earlier, we will write two types of service consumers, one webbased and another windows application based consumer.

Result -

Successfully created simple web service & distributed application.



Lab Assignment - 08

Aim:

Mini Project - Distributed Application for interactive multiplayer games.

Objective:

To create single deployable artifact to run game servers by containerizing the applications.

Theory:

This is the main part of engineering.

Deployment and Orchestration

We have single deployable artifact to run game servers by containerizing the applications. But deploying and maintaining artifacts and services is hard work to do. The Kubernetes helps handle deployments, orchestrate running containers and manage tools.

Load Balancer

The game manager is scalable horizontally, which means that K8s run multiple instances of it concurrently. Therefore, we need load balancer to distribute the requests to game manager pods.

Game Client

I chose Unity3D for game client. The unity game engine uses C# as programming script.



Game Server

The game server is an authoritative server, which means that each client input is processed and validated on the server side, then the game client checks that the server validated data with its predictions.

Game Manager Service

I merged some services with the game manager for the greater good deployment and maintenance thus it must handle a lot of work. I chose go lang to develop the game manager service. The game manager app is a stateless service and could be scaled horizontally.

APIs

An HTTP server APIs to handle the client requests in the game menu and store. A websocket handler to keep a long living connection with the client in the main menu to send events.

Matchmaking

The matchmaking service is responsible for putting the players in a queue and categorizing them depending on latency and rank. The redis fits with these conditions.

Database

A database is needed to store some data like cars, items, game logs and so on, to achieve this, I used MySQL to store relational data.



Kubernetes API

The game manager uses K8S API to create a new pod and run the game server container as a dedicated server.

Game Session Management

The session manager creates, caches, manages and destroys the game sessions.

Event Broker

The services talk to each other using the event broker, like when two players are matched or the game server got ready. I've looked for an opportunity to test the KubeMQ and this situation seemed good for me to use it. I used the KubeMQ community version for this. Also, Kafka was a good option, but on this scale, I prefer not to move forward with it for this project because of RAM consumption and node resource limits to decrease the server costs.

Result -

Distributed Application for interactive multiplayer games.